

❖ الطلب السادس – استراتيجية التخزين المؤقت (Distributed Caching):

- الهدف من هذا المتطلب هو تقليل عدد الاستعلامات المباشرة على قاعدة البيانات عن طريق تخزين البيانات الأكثر طلباً في ذاكرة.
- لماذا نحتاج Caching أصلاً؟
 - في نظام تجارة إلكترونية تحت ضغط عالي، مثلاً كل طلب لـ `/api/products` يترجم إلى:

```
SELECT * FROM products
JOIN categories ...
JOIN inventories ...
WHERE ... LIMIT 15
```

- إذا كان عندنا 100 مستخدم متزامن يطلبون نفس الصفحة، هذا يعني 100 استعلام متطابق على DB في نفس الثانية — وهذا هو الاختناق الأول.
- الحل الهندسي هو وضع طبقة وسيطة تحفظ نتيجة الاستعلام الأول وتعيد استخدامها للطلبات التالية.
- تم اعتماد نمط Cache-Aside وهو النمط الأكثر شيوعاً في أنظمة التجارة الإلكترونية، وفق ما يلي:

الأداة	الدور	السبب
Redis (Docker)	قاعدة بيانات الكاش	سرعة الوصول في الذاكرة
predis/predis	مكتبة اتصال PHP بـ Redis	متوافقة مع Laravel 10
Cache::remember()	دالة في الـ Laravel للكاش	تبسيط منطق Cache-Aside

- لماذا اخترنا Redis تحديداً؟
 - Redis ليست مجرد كاش — هي In-Memory Data Structure Store وهذا يعني أن البيانات موجودة في الـ RAM مباشرة، والوصول إليها لا يحتاج I/O على الهارد.
 - الفرق العملي:

MySQL Query → Disk I/O + CPU parsing + Network ≈ 20-100ms

■ يعني Redis أسرع بـ 100 مرة على الأقل.

● لماذا اخترنا Cache-Aside وليس Read-Through أو Write-Through؟

■ عندنا ثلاثة أنماط رئيسية:

1. Cache-Aside (المطبق): التطبيق هو المسؤول عن القراءة من الكاش والكتابة فيه. بحيث

أنه إذا لم يجد البيانات يجلبها من DB ويحفظها. هذا النمط مناسب لأن البيانات تُقرأ كثيراً وتُعدّل قليلاً.

2. Write-Through: كل كتابة على DB تُكتب أيضاً في الكاش فوراً. مناسب لو الكتابة

والقراءة بنفس الكثرة — لكن في مشروعنا القراءة أكثر بكثير.

3. Read-Through: الكاش نفسه يجيب البيانات من DB تلقائياً. يحتاج library

خاصة ومعقدة أكثر مما نحتاج.

● من المشاكل التي واجهناها هي كالاتي:

■ مستخدم اشترى منتج فأصبحت: DB: quantity = 3. لكن Redis لا يزال يقول أن: quantity = 5

● الحل الذي طبقناه هو مسح المفاتيح المتأثرة فقط من الكاش. بحيث أنه عند عملية الـ checkout

ننتبع كل منتج وتصنيفاته ونحذف مفاتيحه من الـ Redis. ولا نقوم بعملية flush أي إفراغ

لكامل الكاش مما يؤدي إلى مسح كافة البيانات المخزنة. بل نكتفي بمسح المتغير فقط.

● كذلك تم ضبط TTL = 3600 ثانية والتي تعادل ساعة كاملة، لأن بيانات المنتجات لا تتغير

كثيراً، أما لو اخترنا 60 ثانية مثلاً فالكاش سيفقد فائدته. أما لو اخترنا 24 ساعة فقد يرى

المستخدم بيانات قديمة.

● قمنا بتشغيل سيرفر الـ Redis على Docker باستخدام Volume للحفظ الدائم وعدم ضياع

البيانات وفق الإعدادات التالية:

```
docker run -d \
--name redis \
```

```
-p 6379:6379 \  
-v redis_data:/data \  
redis:alpine redis-server --appendonly yes
```

- خيار `--appendonly yes` يضمن حفظ البيانات على الهارد حتى بعد إيقاف الـ

Container.

- أما بالنسبة لإعدادات الـ `env`، فقد تم ضبط ملف `.env` كالآتي:

```
CACHE_DRIVER=redis  
CACHE_STORE=redis  
REDIS_CLIENT=predis  
REDIS_HOST=192.168.146.135  
REDIS_PORT=6379  
REDIS_PASSWORD=null  
REDIS_DB=0  
REDIS_CACHE_DB=0
```

- مثال عملي من المشروع في `ProductController`:

```
private int $cacheTTL = 3600; // كاملة ساعة :الكاش مدة  
  
public function index(Request $request)  
{  
    $cacheKey = "products_" . ($request->category_id ?? "all")  
        . "_page_" . ($request->page ?? 1);  
  
    $products = Cache::remember($cacheKey, $this->cacheTTL, function () use ($request) {  
        $query = Product::with(["category", "inventory"]);  
        if ($request->has("category_id")) {  
            $query->where("category_id", $request->category_id);  
        }  
        return $query->paginate(15);  
    });  
  
    return $this->successResponse(new ProductCollection($products), "...");  
}
```

- عند التعديل `Cache Invalidation`:

```
private function clearProductListCache(?int $categoryId = null): void  
{  
    $totalPages = ceil(Product::count() / 15);
```

```

for ($page = 1; $page <= $totalPages; $page++) {
    Cache::forget("products_all_page_{$page}");
    if ($categoryId) {
        Cache::forget("products_{$categoryId}_page_{$page}");
    }
}
}
}

```

• مفاتيح Redis في مشروعنا :

مفتاح Redis	يمثل	ينشأ في	يُسمح في
product_{id}	منتج واحد	ProductController@show	update / destroy / checkout
products_all_page_{n}	قائمة كل المنتجات	ProductController@index	أي تغيير في المنتجات
products_{cat}_page_{n}	منتجات كاتيجوري محددة	ProductController@index	تغيير منتج في الكاتيجوري
category_{id}	كاتيجوري واحدة	CategoryController@show	update / destroy
categories_all	كل الكاتيجوريز	CategoryController@index	أي تغيير في الكاتيجوريز
inventory_{id}	مخزون منتج	InventoryController@show	تحديث المخزون / checkout

• تم إضافة آلية Fallback تلقائية في AppServiceProvider.php حتى لو كان

Redis لا يعمل لسبب ما، فإن النظام سيستمر بالعمل، وهذا محقق وفق الكود التالي :

```

public function boot(): void
{
    try {
        Cache::store("redis")->get("health_check");
    } catch (Exception $e) {
        Log::warning("Redis unavailable, switching to file cache");
        config(["cache.default" => "file"]);
    }
    // الإعدادات باقي ...
}

```

❖ الطلب السابع – التحكم في الأقفال (Concurrency Control):

- الهدف من هذا المتطلب هو حماية البيانات الحساسة من التعارض عند وصول أكثر من طلب في نفس الوقت إلى نفس المورد، خاصة في العمليات التي قد تؤدي إلى فقدان الاتساق أو إنتاج نتائج غير صحيحة.
- في هذا المشروع تم تطبيق قفلين رئيسيين داخل مسار إنشاء الطلب (checkout). وهما:
 1. قفل على صفوف السلة الخاصة بالمستخدم.
 2. قفل على صفوف المخزون الخاصة بالمنتجات المطلوبة.
- القفلان المستخدمان في المشروع هما من نوع: Pessimistic Locking. بحيث أننا استخدمنا الدالة lockForUpdate() والتي تطلب من قاعدة البيانات حجز الصف فعلياً داخل المعاملة (transaction) حتى تنتهي العملية، وبالتالي فإن أي طلب آخر يحاول تعديل نفس الصف يجب أن ينتظر.

● القفل الأول – قفل السلة:

- تم تطبيق قفل السلة داخل:

app/Http/Controllers/Api/OrderController.php
checkout()

- يبدأ الكود المعاملة أولاً، ثم يقرأ صفوف السلة الخاصة بالمستخدم مع:

```
$cartItems = Cart::with('product')  
->where('user_id', $user->id)  
->lockForUpdate()  
->get();
```

- بحيث أن صفوف السلة تُقفل فعلياً أثناء تنفيذ عملية الشراء، فلا يستطيع طلب آخر تعديل نفس صفوف السلة في الوقت نفسه حتى تنتهي المعاملة.
- تم استخدام هذا القفل لحماية عملية `checkout` من التعارضات المتزامنة، مثل:
 - ✓ قيام المستخدم بإرسال طلبي `checkout` في اللحظة نفسها.
 - ✓ تعديل السلة من نافذة أخرى أو جهاز آخر أثناء إنشاء الطلب.

✓ حذف عنصر من السلة بينما لا تزال عملية إنشاء الطلب تقرأ نفس البيانات.

■ لو لم يكن هذا القفل موجودًا، فقد يقرأ النظام السلة في حالة، ثم تتغير السلة أثناء التنفيذ، وهذا قد يؤدي إلى:

✓ إنشاء طلب غير مطابق للحالة النهائية للسلة.

✓ تكرار عملية الشراء.

✓ سلوك غير متوقع عند التزامن العالي.

■ كيف يعمل هذا القفل عمليًا؟

✓ يبدأ transaction.

✓ يتم قفل صفوف السلة الخاصة بالمستخدم بواسطة lockForUpdate()

✓ يبقى القفل قائمًا حتى commit أو rollback.

✓ إذا وصل طلب ثانٍ لنفس المستخدم في الوقت نفسه، فسيضطر إلى الانتظار حتى ينتهي الطلب الأول، وبذلك نضمن أن بناء الطلب يتم على نسخة ثابتة ومتسقة من السلة.

● القفل الثاني – قفل المخزون:

■ تم تطبيق قفل السلة داخل:

app/Http/Controllers/Api/OrderController.php وتحديدًا داخل الدالة: checkout() داخل الحلقة التي تمر على عناصر السلة كما في الكود التالي:

```
$inventory = \App\Models\Inventory::where('product_id', $item->product_id)
->lockForUpdate()
->first();
```

■ نوع هذا القفل Pessimistic Locking لأن صف المخزون الخاص بالمنتج يتم قفله قبل

فحص الكمية وقبل الخصم منها.

■ لماذا استخدمنا قفل المخزون؟

✓ هذا هو أهم قفل في النظام لأنه يمنع مشكلة Overselling، أي بيع كمية أكبر من

الكمية المتوفرة فعليًا في المخزون.

- ✓ مثال على المشكلة بدون قفل، إذا كانت الكمية المتوفرة لمنتج ما = 5. المستخدم أول يريد شراء 4. المستخدم ثانٍ يريد شراء 3 في اللحظة نفسها
- ✓ إذا قرأ الطالبان الكمية نفسها في الوقت نفسه بدون قفل، فقد يعتبر كل واحد منهما أن الكمية كافية، ثم يتم الخصم مرتين، فتظهر نتيجة غير صحيحة أو سالبة أو يحدث بيع فوق المتاح.
- ✓ أما مع lockForUpdate() فإن الطلب الأول يقفل صف المخزون. والطلب الثاني ينتظر.
- ✓ بعد انتهاء الأول، يقرأ الثاني القيمة الجديدة الحقيقية. وإذا لم تعد الكمية كافية، يتم رفض العملية.

■ كيف يعمل هذا القفل عملياً؟

- ✓ قراءة عنصر من السلة.
- ✓ الوصول إلى سجل المخزون الموافق له.
- ✓ قفل سجل المخزون بـ lockForUpdate().
- ✓ التحقق من أن الكمية كافية.
- ✓ تنفيذ الخصم من المخزون.
- ✓ الاستمرار إلى المنتج التالي.

❖ الطلب الثامن - سلامة المعاملات (Transaction Integrity / ACID):

- الهدف هو ضمان أن العمليات المركبة مثل: دفع + تحديث مخزون + إنشاء طلب إما أن تنجح كلها أو تفشل كلها معاً حتى تحت الوصول المتزامن لمئات المستخدمين.
- في أنظمة التجارة الإلكترونية وتوليد الفواتير، تتكون عملية الشراء أو الدفع الواحدة من مجموعة خطوات متتالية في قاعدة البيانات (مثل: خصم المخزون → إنشاء الفاتورة → خصم رصيد المستخدم → تحديث حالة الطلب).
- الهدف الأساسي من الـ Transaction هو دمج هذه العمليات المتعددة وتحويلها إلى "وحدة عمل واحدة غير قابلة للتجزئة، بحيث إما أن تنجح كل هذه الخطوات معاً ويتم حفظها نهائياً في قاعدة البيانات، أو تفشل خطوة واحدة منها فيقوم النظام بالتراجع عن كل الخطوات السابقة كأن شيئاً لم يكن.
- السيناريو بدون Transaction: يقوم العميل بالضغط على "تأكيد الشراء"، فيقوم الكود بخصم القطعة من المخزون، وأثناء الانتقال لخطوة إنشاء الفاتورة وحفظها، يحدث خطأ في السيرفر (كقطع الاتصال أو امتلاء الذاكرة). بالتالي تكون النتيجة: خصم المنتج من المستودع ولم تصدر له فاتورة، وضاع حق الموقع والعميل
- باستخدام Transaction: بفضل تغليف الكود بـ `DB::beginTransaction()`، لو حدث أي خطأ في خطوة الفاتورة، يتم إطلاق استثناء (Exception) يُجبر النظام على تنفيذ أمر التراجع `DB::rollBack()`، فيعود المخزون كما كان تلقائياً. بحيث تكون تسلسل العمليات كالتالي:

`DB::beginTransaction()`



← مخزون `lockForUpdate + decrement`



← طلب `Order::create`



← عناصر `OrderItem::create`



← سلة `Cart::delete`



- معايير ACID التي يحققها ال Transaction في نظامنا: يعتمد تصميم ال Transaction في هذا المشروع على تحقيق المعايير العالمية الأربعة لسلامة قواعد البيانات والمعروفة بـ ACID Properties:

الخاصية	المعنى	التطبيق في الكود
Atomicity	كل العمليات أو لا شيء	DB::beginTransaction() + rollback()
Consistency	البيانات دائماً في حالة صحيحة	التحقق من المخزون قبل الحفظ
Isolation	العمليات المتزامنة لا تتداخل	lockForUpdate()
Durability	البيانات محفوظة بعد النجاح	DB::commit()

- جدول تغطية ACID في المشروع:

السبب	Lock	Transaction	الدالة
عمليات مركبة + تزامن عالي	lockForUpdate	DB::transaction	checkout()
إنشاء Product + Inventory معاً	لا يوجد	DB::transaction	store() - Product
عملية واحدة فقط	لا يوجد	لا يوجد	update()
عملية واحدة فقط	لا يوجد	لا يوجد	destroy()
Admin فقط، تعارض نادر	لا يوجد	لا يوجد	updateInventory()

- بناءً على ما تقدم، فإن دمج ال Database Transactions مع الأقفال الصارمة (Pessimistic Locking) شكل خط الدفاع البرمجي الأول للنظام، محولاً العمليات الحساسة من مجرد أكواد متتالية إلى منظومة برمجية متماسكة تضمن استقرار الحسابات والمخازن تحت أدق ظروف فحص التزامن وعمليات الضغط المرتفع.

• دور decrement() في سلامة المعاملات:

■ وجدنا مشكلة أثناء الاختبار وفق السيناريو التالي:

✓ PHP يقرأ $quantity = 5$

✓ فجوة زمنية ← مستخدم آخر يقرأ نفس القيمة!

✓ PHP يحسب $3 = 5 - 2$

✓ DB يحفظ $quantity = 3$

✓ DB يحفظ $quantity = 3$ ← مستخدم ثانٍ كتب نفس القيمة!

✓ النتيجة: بعنا نفس القطعة مرتين!

■ لذلك استخدمنا decrement() بدلاً من save() وهو قرار هندسي يرتبط مباشرة بسلامة المعاملات. بحيث أن العملية الحسابية تحدث داخل DB مباشرة — لا فجوات، لا PHP في المنتصف.

■ الطريقة القديمة save() غير آمنة:

```
// فقط الذاكرة في تغيير (في PHP في 1 خطوة)
$inventory->quantity -= $item->quantity;

... هنا أخرى عمليات تمر ...

// وقت بعد (في DB في 2 خطوة)
$inventory->save(); // قديمة تكون قد الذاكرة في القيمة: خطر
```

■ الطريقة الجديدة decrement() آمنة أكثر:

```
// البيانات قاعدة في مباشرة واحدة عملية
$inventory->decrement("quantity", $item->quantity);

// إلى SQL:
// UPDATE inventories
// SET quantity = quantity - 2
// WHERE product_id = 5
// (بعد يخرج ولم القفل داخل)
```

▪ الميزات الهندسية ل decrement():

✓ أولاً: Atomicity على مستوى ال SQL، بحيث أن العملية لا تتجزأ على مستوى المحرك. لا يمكن لأي شيء أن يتدخل بينها.

✓ ثانياً: Round Trips. فعند استعمال save() ستتم عملية SELECT ثم UPDATE. بالتالي عمليتين في ال DB. أما باستخدام decrement() فستتم عملية ال UPDATE مباشرة. بالتالي تقليل الضغط على ال DB.

● الخلاصة النهائية في في تابع ال checkout():

▪ Transaction يضمن All or Nothing

▪ lockForUpdate يمنع القراءة المتزامنة

▪ decrement يضمن العملية الحسابية Atomic

❖ الطلب التاسع – اختبار الاستقرار تحت الضغط:

- الهدف من هذا المتطلب هو التحقق من قدرة النظام على خدمة عدد كبير من المستخدمين المتزامنين، وتحليل سلوك التطبيق عند زيادة الحمل، وتوثيق الاختناقات، والمشكلات، والحلول التي تم تطبيقها لتحسين النتيجة.
 - تم تنفيذ عدة اختبارات من خلال أداة k6 على مجموعة من ال endpoints الموجودة في مشروعنا بهدف قياس قدرة التحمل وتحديد الاختناقات.
 - في جميع الاختبارات تمت محاكاة 100 مستخدم حاليين في نفس اللحظة. بحيث تبدأ السكريبت بالتزايد تدريجياً حتى الوصول إلى 100 مستخدم، ثم الهبوط تدريجياً لإنهاء الاختبار.
 - العوائق التي واجهناها أثناء تجربة الاختبارات:
1. تشغيل السيرفر عبر php artisan serve. وهو سيرفر محدود الإمكانيات مناسب لعمليات التطوير ولا يحاكي البيئة الواقعية. بالتالي قمنا بنقل المشروع إلى ملفات السيرفر الأساسية وقمنا بتشغيله من ك Virtual Host.
 2. ال Ratelimit الموجود على ال endpoints تحقيقاً للمتطلب الثاني في المشروع. لذلك قمنا بتعطيله مبدئياً حتى نستطيع التجربة ومحاكاة كم كبير من الطلبات من نفس ال IP.
 3. الحمل الإضافي الذي تقوم به أداة Telescope المسؤولة عن مراقبة الموارد مما يؤدي إلى زيادة محدودية السيرفر. لذلك قمنا بإيقافها مؤقتاً.
 4. أخطاء تحدث بسبب التعامل مع ال Redis ومشاكل في الاتصال به عند الحمولة الكبيرة والزائدة. لذلك قمنا بإيقافه وتحويل التخزين إلى file أثناء التجربة.
- **الاختبار الأول: GET /api/products وهي المسؤولة عن جلب المنتجات:**
 - بدأنا بهذا ال api البسيط للتدرج في مستوى الاختبار وسد كل ثغرات النقص وتطبيق محاكاة واقعية وتجاوز كل العقبات كما سنرى.
 - تم إنشاء ملف السكريبت المنفصل ضمن المشروع باسم: stress-products.js
 - ضمن السكريبت تم إضافة عدادات لمعرفة حالات كود الاستجابة. بحيث تعطي نتيجة أوضح وأدق.

- نقوم بتشغيل الملف بالتعليمة التالية:

```
& C:\Progra~1\k6\k6.exe run -e BASE_URL=http://ecommerce.local/api stress-products.js
```

بعد تجاوزنا للعوائق السابقة، استطعنا تحقيق نسبة نجاح 100% في هذه ال endpoint كما في الصورة التالية:

```
TOTAL RESULTS

checks_total.....: 1471    10.395001/s
checks_succeeded...: 100.00% 1471 out of 1471
checks_failed.....: 0.00%   0 out of 1471

✓ status is 200

CUSTOM
rate_4xx.....: 0.00%   0 out of 1471
rate_5xx.....: 0.00%   0 out of 1471
request_ok.....: 100.00% 1471 out of 1471
status_200.....: 1471    10.395001/s
status_2xx.....: 1471    10.395001/s
```

● الاختبار الثاني: `Post /api/checkout` وهي المسؤولة عن إنشاء الطلب:

- في هذا السيناريو سنقوم بمحاكاة 100 مستخدم يقومون بعمليات إنشاء الطلبات بشكل متزامن. بحيث في البداية سنقوم بتسجيل وتسجيل دخول 100 مستخدم لمرة واحدة فقط، لأن الهدف هو تجربة سيناريو إنشاء الطلب وليس غيرها من العمليات، ثم سنستخدم ال Token التي سنحصل عليها في محاكاة السيناريو.

- قمنا بإنشاء ملف سكريبت لهذا الاختبار باسم: `stress-checkout.js`.

- الخطوات التي ينفذها كل مستخدم افتراضي هي كالآتي:

✓ استخدام token جاهز تم توليده عبر الدالة `setup()`

✓ انتظار قصير بعد بدء الجلسة.

✓ انتظار منطقي قبل الإضافة إلى السلة، وتم تقديره بـ 1.5 ثانية.

✓ إرسال الطلب. ثم انتظار مدة 0.8 ثانية.

✓ استقبال الطلب. ثم انتظار 1.5 ثانية.

✓ إرسال عملية الشراء النهائية ثم انتظار 0.8 ثانية.

✓ استقبال معرف الطلب.

- هذه القيم لا تمثل "تخفيفًا مصطنعًا" للاختبار، بل تمثل: Think Time. أي الزمن الطبيعي الذي يحتاجه المستخدم بين خطوة وأخرى أثناء الشراء.
- يتم تشغيل الملف عبر التعليمات:

```
& C:\Progra~1\k6\k6.exe run `
-e BASE_URL=http://ecommerce.local/api `
-e USER_COUNT=100 `
-e TARGET_USERS=100 `
-e SINGLE_FLOW_MODE=true `
-e PREAUTH_IN_SETUP=true `
-e CLEAN_CART_FIRST=false `
-e SLEEP_AFTER_LOGIN=0.8 `
-e SLEEP_BEFORE_CART_ADD=1.5 `
-e SLEEP_AFTER_CART_ADD=0.8 `
-e SLEEP_AFTER_CART_VIEW=1.5 `
-e SLEEP_AFTER_CHECKOUT=0.8 `
-e PRODUCT_POOL_SIZE=500 `
-e MAX_DURATION=15m `
stress-checkout.js
```

- عند تجربة الاختبار حصلنا على النتائج التالية:

```
- `checkout_ok` = 97%`
- `flow_ok` = 93%`
- `order_verify_ok` = 94%`
- `http_req_failed` = 1.34%`
- الكلي HTTP عدد طلبات : `597`
- عدد عمليات الشراء الناجحة :
  - `checkout_201` = 97`
- `checkout` : عدد حالات الفشل الوظيفي في :
  - `checkout_400` = 1`
  - `checkout_500` = 2`
```

- كما حصلنا على هذه الأزمات:

```
- `cart_add_duration` avg = 25.36s`
- `cart_view_duration` avg = 13.94s`
- `checkout_duration` avg = 10.63s`
```

```
- `order_verify_duration avg = 4.26s`  
- `http_req_duration p95 = 30.63s`  
- `iteration_duration avg = 1m2s`
```

■ لماذا لم نصل إلى نسبة 100%؟ قمنا بقراءة السجلات في laravel.log فوجدنا المشاكل في:

✓ Redis - البعيدة عبر الشبكة

✓ وتأثيرها على JWT blacklist/cache checks

● اختبار الضغط على **Post /api/register** أي عملية تسجيل الحساب:

■ قمنا بإنشاء ملف سكريبت منفصل باسم stress-register.js.

■ حتى لا تفشل الطلبات بسبب تكرار البريد الإلكتروني، تم توليد email مختلفة لكل مستخدم

باستخدام توليد رقم عشوائي في الإيميل.

■ يمكننا تشغيل السكريبت من خلال التعليمة التالية:

```
& C:\Progra~1\k6\k6.exe run `  
-e BASE_URL=http://ecommerce.local/api `  
-e TARGET_USERS=100 `  
-e MAX_DURATION=10m `  
stress-register.js
```

■ في الجولة الأساسية على 100 مستخدم متزامن ظهرت النتائج التالية:

```
- `register_ok = 100.00%`  
- `token_issued_ok = 100.00%`  
- `status_201 = 100`  
- `checks_succeeded = 100.00%`  
- `http_req_failed = 0.00%`
```

■ والنتائج الزمنية كالآتي:

```
- `register_duration avg = 21.56s`  
- `register_duration med = 22.28s`  
- `register_duration p90 = 26.32s`  
- `register_duration p95 = 26.73s`  
- `register_duration max = 27.05s`
```

❖ الطلب العاشر – القياس وتحديد الاختناقات (Benchmarking & Bottleneck)

(Identification)

- في هذا الطلب قمنا بما يلي:
 - قياس أداء واجهات الـ API بشكل فعلي ومنهجي.
 - تجميع نتائج الأداء في صورة قابلة للتحليل.
 - تحديد نقاط الاختناق الحقيقية داخل النظام.
 - ربط النتائج بالأسباب البرمجية أو المعمارية داخل المشروع.
- هذا المتطلب لا يكفي بتشغيل اختبار ضغط عشوائي، بل يتطلب بناء طريقة عملية تجعلنا نجيب عن الأسئلة التالية:
 - أي API هي الأبطأ؟
 - هل البطء سببه قاعدة البيانات أم التطبيق أم التخزين المؤقت أم الشبكة؟
 - ما عدد الاستعلامات التي تُنفذ داخل كل طلب؟
 - ما حجم الاستجابة التي يعيدها كل endpoint ؟
 - ما التعديلات التي أُضيفت داخل المشروع لكي يصبح القياس قابلاً للتكرار والتحليل؟
- في هذا المتطلب تم بناء طبقة قياس كاملة داخل المشروع، ثم تم تنفيذ اختبارات فعلية على أغلب واجهات الـ API العامة والمحمية ومسارات الإدارة، ثم تم استخراج النتائج وتحليلها.
- تم إجراء الخطوات العملية التالية:
 - تم إنشاء طبقة Middleware تعمل على كل طلب الـ api.
 - تم إنشاء قناة logging مستقلة تحفظ نتائج القياس في ملف خاص.
 - تم إنشاء أمر Artisan يقرأ السجل ويخرج ملخصاً مرتباً.

■ تم إنشاء سكريبت k6 وسكريبت PowerShell لتعبئة بيانات وهمية لنتمكن من القياس بشكل مبدئي.

● الملفات التي تمت إضافتها:

■ ملف **config/benchmark.php**: هذا الملف أُضيف ليكون مركز إعدادات القياس، ويحتوي على:

✓ تفعيل أو تعطيل القياس.

✓ اسم قناة السجل المستخدمة.

✓ حد اعتبار الطلب بطيئًا.

✓ حد اعتبار الاستعلام بطيئًا.

■ ملف

app/Http/Middleware/ApiBenchmarkMiddleware.php

هذا الملف هو الذي جعل المشروع يسجل بيانات قابلة للتحليل بدل الاعتماد على الانطباع اليدوي. بحيث وظيفته:

✓ اعتراض كل طلب API.

✓ قياس زمن الطلب من البداية للنهاية.

✓ تشغيل `DB::enableQueryLog()` أثناء الطلب.

✓ حساب المتغيرات التالية:

```
- `duration_ms`  
- `query_count`  
- `db_time_ms`  
- `db_share_percent`  
- `max_query_ms`  
- `max_query_sql`  
- `peak_memory_mb`  
- `response_kb`
```

✓ تخزين النتيجة في ملف السجل.

✓ توليد `bottleneck_hint` أولي.

■ تعديل **app/Http/Kernel.php**: تم تعديل هذا الملف لجعل جميع الـ apis تمر عبر الـ Middleware التي قمنا بإنشائها.

■ تعديل **config/logging.php**: تم تعديل هذا الملف لإضافة قناة جديدة اسمها: **benchmark** وظيفتها تسجيل قياسات الأداء في ملف مستقل لكل يوم بيومه وبتنسيق معين قمنا باختياره.

■ تعديل

app/Console/Commands/ApiBenchmarkReportComm

and.php: بحيث قمنا بإضافة أمر التشغيل إلى هذا الملف لكي نستطيع تشغيل تعليمة تقوم بتحليلي السجلات التي تم تسجيلها. بحيث يمكن تشغيل التعليمة كما الآتي:

```
php artisan benchmark:api-report --limit=20
```

■ إنشاء ملف **benchmark-api.js**: وظيفته إرسال طلبات لأي endpoint نختارها بإعدادات معينة. بحيث نقوم بتشغيل التعليمة كما المثال التالي:

```
$env:BASE_URL='http://localhost/api'  
$env:ENDPOINT='/products'  
$env:VUS='5'  
$env:DURATION='10s'  
$env:SLEEP_SECONDS='0.5'  
& 'C:\Progra~1\k6\k6.exe' run .\benchmark-api.js
```

بحيث أن الملف السابق مناسب لتجربة المسارات التالية:

```
- `GET /api/products`  
- `GET /api/products/1`  
- `GET /api/categories`  
- `GET /api/categories/1`
```

- إنشاء ملف **run-full-api-benchmark.ps1**: هذا الملف أضيف ليكون ملف الاختبار الشامل لجميع الـ endpoints الموجودة في النظام عندنا. بحيث يمكننا تشغيله بالتعليمة التالية:

```
.\run-full-api-benchmark.ps1
```

بحيث أن هذا الملف مسؤول عن تجربة المسارات التالية:

```
- `POST /api/register`  
- `POST /api/login`  
- `POST /api/refresh`  
- `GET /api/me`  
- `POST /api/cart`  
- `GET /api/cart`  
- `PUT /api/cart/{id}`  
- `POST /api/checkout`  
- `GET /api/orders`  
- `GET /api/orders/{id}`  
- `POST /api/logout`  
- `POST /api/categories`  
- `PUT /api/categories/{id}`  
- `DELETE /api/categories/{id}`  
- `POST /api/products`  
- `PUT /api/products/{id}`  
- `DELETE /api/products/{id}`  
- `PUT /api/inventory/{productId}`
```

- تعديل ملف **env**: بحيث تم التعديل عليه لإضافة مجموعة من الإعدادات المتعلقة بهذا الأمر والتي هي:

```
API_BENCHMARKING_ENABLED=true  
API_BENCHMARK_LOG_CHANNEL=benchmark  
API_BENCHMARK_SLOW_REQUEST_MS=800  
API_BENCHMARK_SLOW_QUERY_MS=250
```

● كيف يعمل القياس داخليًا؟

- يبدأ عد الزمن باستخدام `.hrtime`.
- يتم تفعيل `.DB::enableQueryLog()`.
- يمر الـ `api` إلى التابع الذي يطلبه.

■ بعد انتهاء العملية والتنفيذ يتم ما يلي:

✓ يتم جمع الاستعلامات المنفذة.

✓ حساب مجموع زمن DB.

✓ تحديد أبطأ استعلام.

✓ حساب حجم الاستجابة.

✓ حساب نسبة DB من الزمن الكلي.

- الآن لكي نستطيع استخراج التقرير الذي نريده والذي يحدد القيم التي تم تسجيلها في الـ log نقوم بالتعليمة التالية:

```
php artisan benchmark:api-report --limit=20
```

- بعد تشغيل أدوات القياس والتحليل، كانت أبرز النتائج كما يلي:

```
PS C:\Users\USER\Desktop\Ecommerce> php artisan benchmark:api-report --limit=20
```

API Benchmark Summary

Samples: 3074

Endpoints: 21

Slowest Endpoints

Endpoint	Samples	Avg ms	DB ms	DB %	Queries	Resp KB	Errors %	Hint
POST api/login	519	4621.81	236.61	3.99	1	0.43	0	One or more slow queries are likely limiting performance.
POST api/cart	434	3376.41	446.54	15.59	6.94	0.16	0.92	One or more slow queries are likely limiting performance.
POST api/checkout	390	2664.38	567.9	21.59	14.69	0.22	2.56	One or more slow queries are likely limiting performance.
GET api/cart	646	2146.7	237.82	12.48	2.63	0.15	0.77	One or more slow queries are likely limiting performance.
GET api/orders/{id}	362	1116.21	181.72	16.2	4	0.82	0	One or more slow queries are likely limiting performance.
POST api/register	632	1054	92.41	4.55	2	0.63	0	One or more slow queries are likely limiting performance.
DELETE api/cart/{id}	2	245.65	39.44	31.15	3	0.07	0	No major bottleneck detected from current samples.
POST api/refresh	5	124.04	7.26	9	1	0.44	0	No major bottleneck detected from current samples.
POST api/products	3	112.6	14.04	12.05	5	0.3	0	No major bottleneck detected from current samples.
PUT api/products/{id}	3	102.63	6.8	6.6	5	0.33	0	No major bottleneck detected from current samples.
POST api/categories	3	100.47	7.53	7.71	3	0.28	0	No major bottleneck detected from current samples.
GET api/orders	5	98.71	12.43	12.64	5	2.77	0	No major bottleneck detected from current samples.
PUT api/cart/{id}	5	92.39	10.35	11.97	6	0.15	0	No major bottleneck detected from current samples.
PUT api/categories/{id}	3	91.13	6.91	7.7	4	0.3	0	No major bottleneck detected from current samples.
PUT api/inventory/{productId}	3	80.83	6.62	8.22	4	0.19	0	No major bottleneck detected from current samples.
DELETE api/categories/{id}	3	76.3	10.04	15.03	3	0.07	0	No major bottleneck detected from current samples.
DELETE api/products/{id}	3	75.39	10.02	13.11	4	0.07	0	No major bottleneck detected from current samples.

GET api/me	5	61.85	6.85	10.77	1	0.25	0	No major bottleneck detected from current samples.
POST api/logout	5	59.24	5.22	8.1	1	0.06	0	No major bottleneck detected from current samples.
GET api/products	22	50.3	4.24	7.65	2.91	1.31	0	No major bottleneck detected from current samples.

Most Database-Bound Endpoints

Endpoint	Samples	Avg ms	DB ms	DB %	Queries	Resp KB	Errors %	Hint
DELETE api/cart/{id}	2	245.65	39.44	31.15	3	0.07	0	No major bottleneck detected from current samples.
POST api/checkout	390	2664.38	567.9	21.59	14.69	0.22	2.56	One or more slow queries are likely limiting performance.
GET api/orders/{id}	362	1116.21	181.72	16.2	4	0.82	0	One or more slow queries are likely limiting performance.
POST api/cart	434	3376.41	446.54	15.59	6.94	0.16	0.92	One or more slow queries are likely limiting performance.
DELETE api/categories/{id}	3	76.3	10.04	15.03	3	0.07	0	No major bottleneck detected from current samples.
DELETE api/products/{id}	3	75.39	10.02	13.11	4	0.07	0	No major bottleneck detected from current samples.
GET api/orders	5	98.71	12.43	12.64	5	2.77	0	No major bottleneck detected from current samples.
GET api/cart	646	2146.7	237.82	12.48	2.63	0.15	0.77	One or more slow queries are likely limiting performance.
POST api/products	3	112.6	14.04	12.05	5	0.3	0	No major bottleneck detected from current samples.
PUT api/cart/{id}	5	92.39	10.35	11.97	6	0.15	0	No major bottleneck detected from current samples.
GET api/products/{id}	21	47.54	5.31	10.89	3	0.76	0	No major bottleneck detected from current samples.
GET api/me	5	61.85	6.85	10.77	1	0.25	0	No major bottleneck detected from current samples.
POST api/refresh	5	124.04	7.26	9	1	0.44	0	No major bottleneck detected from current samples.
PUT api/inventory/{productId}	3	80.83	6.62	8.22	4	0.19	0	No major bottleneck detected from current samples.
POST api/logout	5	59.24	5.22	8.1	1	0.06	0	No major bottleneck detected from current samples.
POST api/categories	3	100.47	7.53	7.71	3	0.28	0	No major bottleneck detected from current samples.
PUT api/categories/{id}	3	91.13	6.91	7.7	4	0.3	0	No major bottleneck detected from current samples.
GET api/products	22	50.3	4.24	7.65	2.91	1.31	0	No major bottleneck detected from current samples.
PUT api/products/{id}	3	102.63	6.8	6.6	5	0.33	0	No major bottleneck detected from current samples.
POST api/register	632	1054	92.41	4.55	2	0.63	0	One or more slow queries are likely limiting performance.

Highest Error Rate Endpoints

Endpoint	Samples	Avg ms	DB ms	DB %	Queries	Resp KB	Errors %	Hint
POST api/checkout	390	2664.38	567.9	21.59	14.69	0.22	2.56	One or more slow queries are likely limiting performance.
POST api/cart	434	3376.41	446.54	15.59	6.94	0.16	0.92	One or more slow queries are likely limiting performance.
GET api/cart	646	2146.7	237.82	12.48	2.63	0.15	0.77	One or more slow queries are likely limiting performance.
POST api/register	632	1054	92.41	4.55	2	0.63	0	One or more slow queries are likely limiting performance.
POST api/login	519	4621.81	236.61	3.99	1	0.43	0	One or more slow queries are likely limiting performance.
POST api/refresh	5	124.04	7.26	9	1	0.44	0	No major bottleneck detected from current samples.
GET api/me	5	61.85	6.85	10.77	1	0.25	0	No major bottleneck detected from current samples.
PUT api/cart/{id}	5	92.39	10.35	11.97	6	0.15	0	No major bottleneck detected from current samples.
GET api/orders	5	98.71	12.43	12.64	5	2.77	0	No major bottleneck detected from current samples.
GET api/orders/{id}	362	1116.21	181.72	16.2	4	0.82	0	One or more slow queries are likely limiting performance.
POST api/logout	5	59.24	5.22	8.1	1	0.06	0	No major bottleneck detected from current samples.
POST api/categories	3	100.47	7.53	7.71	3	0.28	0	No major bottleneck detected from current samples.
PUT api/categories/{id}	3	91.13	6.91	7.7	4	0.3	0	No major bottleneck detected from current samples.
POST api/products	3	112.6	14.04	12.05	5	0.3	0	No major bottleneck detected from current samples.
PUT api/products/{id}	3	102.63	6.8	6.6	5	0.33	0	No major bottleneck detected from current samples.
PUT api/inventory/{productId}	3	80.83	6.62	8.22	4	0.19	0	No major bottleneck detected from current samples.
DELETE api/products/{id}	3	75.39	10.02	13.11	4	0.07	0	No major bottleneck detected from current samples.

DELETE api/categories/{id}	3	76.3	10.04	15.03	3	0.07	0	No major bottleneck detected from current samples.	
GET api/products	22	50.3	4.24	7.65	2.91	1.31	0	No major bottleneck detected from current samples.	
GET api/products/{id}	21	47.54	5.31	10.89	3	0.76	0	No major bottleneck detected from current samples.	

- **Slowest Endpoints** جدول يوضح أبطأ المسارات من حيث الزمن الكلي. هنا نرى بوضوح أن أبطأ المسارات هي:

```
POST /api/login
POST /api/cart
POST /api/checkout
GET /api/cart
GET /api/orders/{id}
POST /api/register
```

هذا يعني أن المسارات الثقيلة ليست مسارات القراءة العامة مثل products، بل المسارات التي تعتمد على المصادقة والكتابة على قاعدة البيانات ومنطق الأعمال.

- **Most Database-Bound Endpoints** جدول يوضح المسارات التي تستهلك نسبة أعلى من وقتها داخل قاعدة البيانات. بحيث أن ترتيب النتائج كالآتي:

```
POST /api/checkout
GET /api/orders/{id}
POST /api/cart
GET /api/cart
```

هذا يؤكد أن جزءاً كبيراً من ببطء عمليات الشراء والسلة ناتج من الاستعلامات المتعددة والعلاقات والكتابة داخل الجداول.

- **Highest Error Rate Endpoints** جدول يوضح المسارات التي تُظهر أخطاء أكثر من غيرها. وهذا هو ترتيبها:

```
POST /api/checkout
POST /api/cart
GET /api/cart
```

هذا مهم جداً لأن المسارات نفسها ليست فقط بطيئة، بل أيضاً هي الأكثر عرضة للفشل، وبالتالي فهي تمثل الاختناق التشغيلي الأساسي في النظام.

• تحليل كل Endpoint على حدى:

POST /api/login

هذا هو أبطأ endpoint في التقرير بمتوسط ms 4621.81 مع 519 عينة، لكن DB ms = 236.61 و DB % = 3.99% فقط مع استعلام واحد تقريبًا. هذا يعني أن الاختناق المتوقع ليس في قاعدة البيانات نفسها، بل في التحقق من كلمة المرور المشفرة Hash::check وإنشاء JWT token، وربما التفاعل مع طبقة الكاش أو ال blacklist الخاصة بالمصادقة. لذلك فبطء login أقرب إلى كلفة المصادقة والتشفير والتوكن، وليس إلى بقاء SQL.

POST /api/cart

هذا المسار بمتوسط ms 3376.41 مع 434 عينة ونسبة DB % = 15.59 ومتوسط 6.94 استعلامات، مع نسبة أخطاء 0.92%. الاختناق المتوقع هنا هو مزيج من استعلامات قاعدة البيانات ومنطق التحقق من المنتج والمخزون ثم updateOrCreate في السلة، إضافة إلى أثر المصادقة قبل الدخول إلى المسار. هذا endpoint ثقيل لأنه لا ينفذ عملية واحدة فقط، بل يجمع بين التحقق والقراءة والكتابة.

POST /api/checkout

هذا أهم endpoint اختناقي في النظام. متوسطه ms 2664.38، وزمن DB داخله 567.9 ms بنسبة 21.59%، مع 14.69 استعلامًا ونسبة أخطاء 2.56% وهي الأعلى في التقرير. نقطة الاختناق المتوقعة هنا هي ال transaction، الأقفال lockForUpdate، التحقق من السلة، تعديل المخزون، إنشاء الطلب وعناصر الطلب، حذف السلة، ثم تنظيف الكاش وإرسال ال job. هذا المسار يمثل اختناقًا حقيقيًا في منطق الأعمال، وليس فقط بطئًا في الاستجابة.

GET /api/cart

متوسطه ms 2146.7 مع 646 عينة و DB % = 12.48 ونسبة أخطاء 0.77%. الاختناق المتوقع هنا هو تحميل بيانات السلة مع المنتجات المرتبطة بها، إضافة إلى كلفة المصادقة JWT قبل الوصول إلى البيانات. وبما أن عدد الاستعلامات متوسط (2.63) لكن الزمن مرتفع، فهذا يشير إلى أن المشكلة ليست SQL فقط، بل أيضًا المصادقة وال serialization والاستجابة.

GET /api/orders/{id}

متوسطه ms 1116.21 مع 362 عينة ونسبة $DB \% = 16.2$ وأربع استعلامات تقريبًا. الاختناق المتوقع هنا هو تحميل الطلب مع العلاقات items.product والتحقق من ملكية المستخدم للطلب. هذه النسبة تعني أن قاعدة البيانات تؤثر بشكل واضح، لكن ليس وحدها؛ هناك أيضًا كلفة تحميل العلاقات وتحويلها إلى JSON.

POST /api/register

متوسطه ms 1054 مع 632 عينة، لكن $DB \% = 4.55$ فقط واستعلامان فقط، ونسبة الأخطاء 0%. هذا يشير بوضوح إلى أن الاختناق المتوقع ليس في SQL، بل في تشفير كلمة المرور Hash::make وإنشاء JWT token بعد التسجيل.

DELETE /api/cart/{id}

هذا endpoint لديه عینتان فقط، ومتوسط ms 245.65 ونسبة $DB \% = 31.15$. هذه أعلى نسبة DB في التقرير، لكن عدد العينات قليل جدًا فلا يمكن بناء حكم قوي. مبدئيًا نقطة الاختناق المتوقعة هي عملية إيجاد العنصر المرتبط بالمستخدم ثم حذفه من قاعدة البيانات، لكن هذا يحتاج بيانات أكثر للتأكيد.

POST /api/refresh

متوسطه ms 124.04 مع 5 عينات و $DB \% = 9$. الاختناق المتوقع هنا ليس قاعدة البيانات، بل منطق JWT refresh نفسه والتعامل مع التوكن الحالي وإصدار توكن جديد. ولأن العينات قليلة فالنتيجة تبقى إشارة أولية فقط.

POST /api/products

متوسطه ms 112.6 مع 3 عينات فقط و 5 استعلامات. الاختناق المتوقع هنا هو إنشاء المنتج ثم إنشاء سجل inventory ثم تنظيف الكاش المرتبط بالقوائم. النتيجة لا تشير إلى اختناق كبير، لكن عدد العينات قليل جدًا.

PUT /api/products/{id}

متوسطه 102.63 ms مع 3 عينات. الاختناق المتوقع هنا هو تحديث المنتج ثم مسح الكاش المرتبط بالمنتج والقوائم. لا يوجد دليل على اختناق مهم، والنتيجة تشير إلى أن هذا المسار مقبول زمنيًا في الوضع الحالي.

POST /api/categories

متوسطه 100.47 ms مع 3 عينات. الاختناق المتوقع هنا بسيط جدًا، وهو إنشاء category وحفظها في قاعدة البيانات. لا يظهر هنا عنق زجاجة واضح.

GET /api/orders

متوسطه 98.71 ms مع 5 عينات و 5 استعلامات وحجم استجابة 2.77 KB. الاختناق المتوقع هنا هو جلب قائمة الطلبات مع العلاقات والـ pagination. لا يوجد اختناق واضح حاليًا، لكن لو زاد عدد الطلبات لكل مستخدم قد يرتفع زمنه لاحقًا.

PUT /api/cart/{id}

متوسطه 92.39 ms مع 5 عينات و 6 استعلامات. الاختناق المتوقع هنا هو تحميل عنصر السلة، تحميل المنتج والمخزون، التحقق من الكمية المتاحة، ثم تحديث الكمية. لا يظهر حاليًا أنه اختناق رئيسي، لكنه منطقيًا أكثر كلفة من endpoints القراءة البسيطة.

PUT /api/categories/{id}

متوسطه 91.13 ms مع 3 عينات. الاختناق المتوقع منخفض جدًا، وغالبًا هو عملية التحديث المباشرة في قاعدة البيانات فقط.

PUT /api/inventory/{productId}

متوسطه 80.83 ms مع 3 عينات و 4 استعلامات. الاختناق المتوقع هو تحديث المخزون ثم تنظيف الكاش المرتبط بالمنتج أو المخزون. لا توجد إشارة حاليًا إلى اختناق مؤثر.

DELETE /api/categories/{id}

متوسطه 76.3 ms مع 3 عينات. الاختناق المتوقع هو الحذف المباشر وربما أثر القيود المرجعية أو العلاقات إن وجدت، لكن لا توجد إشارة قوية إلى مشكلة.

DELETE /api/products/{id}

متوسطه 75.39 ms مع 3 عينات. الاختناق المتوقع هو حذف المنتج ثم مسح الكاش المرتبط به . لا يبدو أنه يمثل مشكلة أداء.

GET /api/me

متوسطه 61.85 ms مع 5 عينات واستعلام واحد فقط. الاختناق المتوقع هنا ليس قاعدة البيانات، بل المصادقة JWT واستخراج المستخدم الحالي . وهو endpoint خفيف نسبيًا.

POST /api/logout

متوسطه 59.24 ms مع 5 عينات واستعلام واحد. الاختناق المتوقع هو إبطال التوكن أو وضعه في blacklist ، وليس قاعدة البيانات. المسار يبدو مقبول الأداء.

GET /api/products

متوسطه 50.3 ms مع 22 عينة فقط و $DB \% = 7.65$ هذا مسار جيد جدًا مقارنة ببقية النظام، والاختناق المتوقع فيه منخفض. هذا يؤكد أن الكاش على قوائم المنتجات فعّالة نسبيًا، وأن endpoint القراءة العامة ليست نقطة الاختناق الأساسية.

GET /api/products/{id}

ظهر في جدول Most Database-Bound بمتوسط 47.54 ms و 21 عينة و $DB \% = 10.89$. هذه أيضًا نتيجة جيدة، وتشير إلى أن الكاش على المنتج المفرد يعمل جيدًا، ولا يوجد اختناق كبير هنا .